HCCL软件架构和算子编程模型



CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V5.0	2026/8/30	张兴平	全部	初版内容

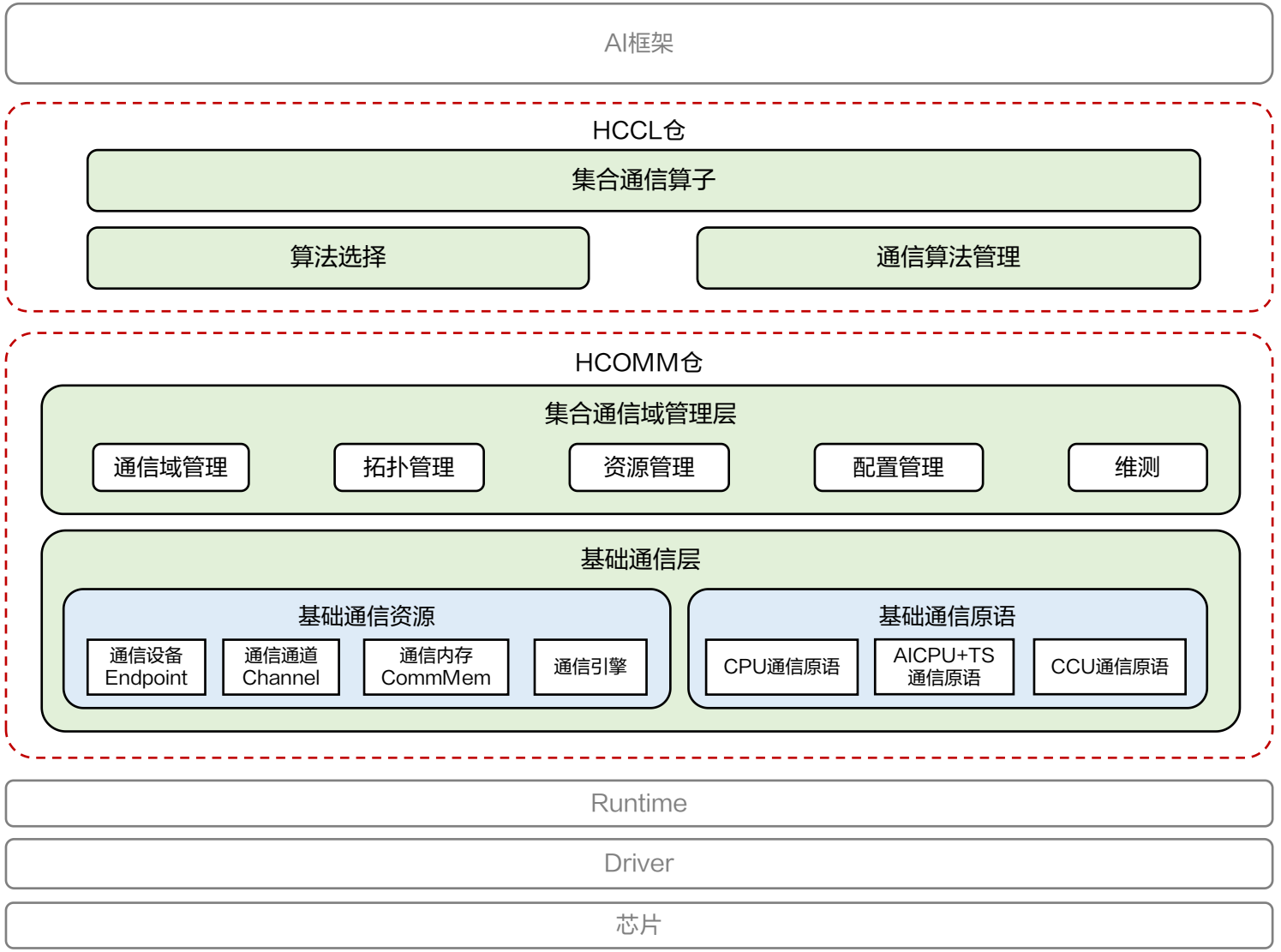
目录

Part 1 HCCL软件架构

Part 2 HCCL算子编程模型

Part 3 HCCL算子常用编程API介绍

HCCL整体软件架构



设计动机：

- 提供自定义算子/算法开发能力；
- 基础通信能力与业务场景解耦；
- 控制面与数据面解耦；
- 抽象基础通信能力，屏蔽硬件差异。

HCCL仓：

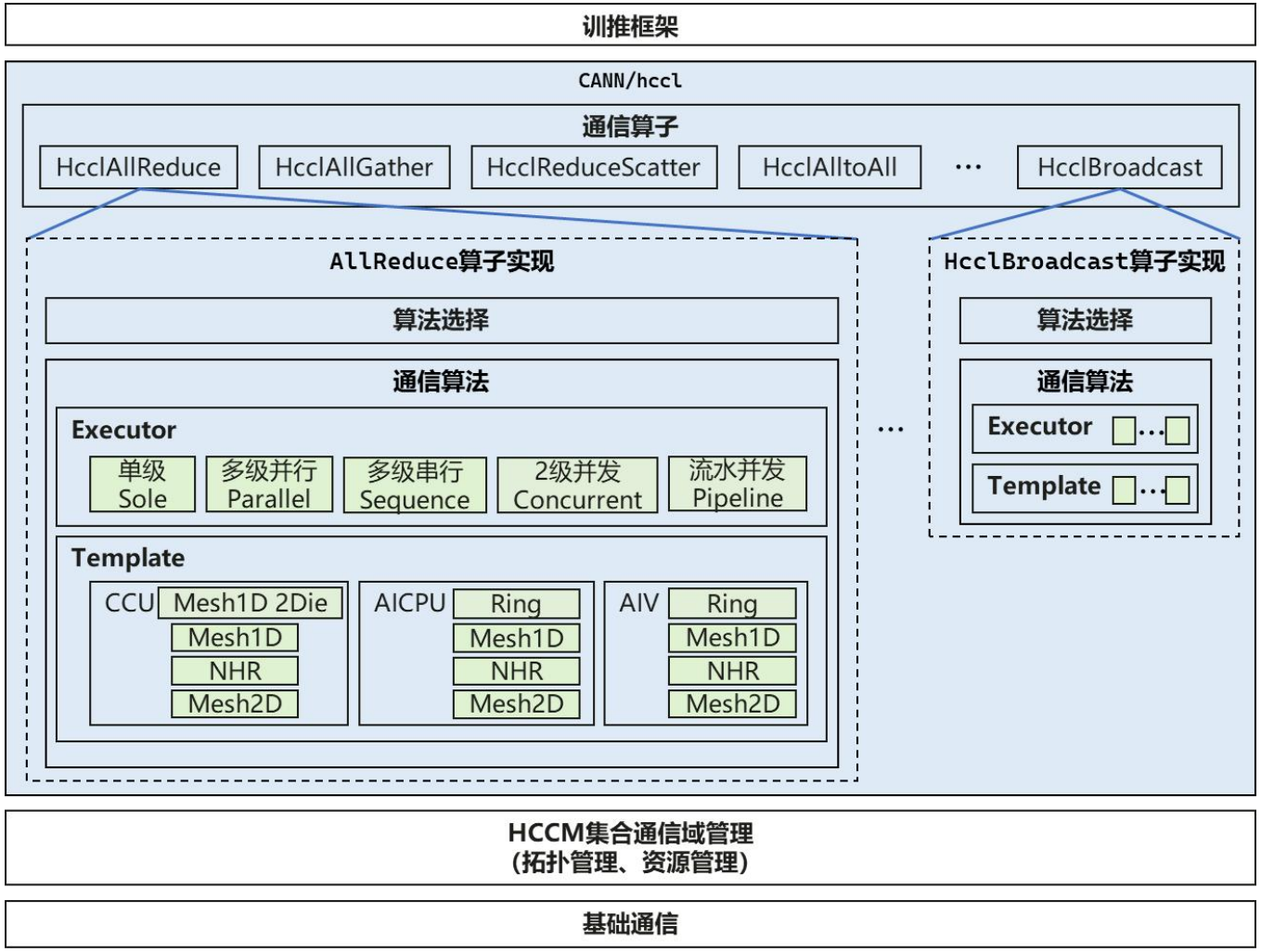
- 集合通信算子入口与实现；
- 算子实现包含：
  - 通信算法管理：管理算子的多种算法实现
  - 算法选择：根据通信域、算子信息、拓扑等选择合适的算法

HCOMM仓：

- 基础通信：提供基础通信能力
  - 基础通信资源：创建、销毁能力；
  - 基础通信原语：数据面编程能力；
- 集合通信域管理：面向集合通信场景的业务管理能力。

https://gitcode.com/cann/hccl/blob/master/docs/zh/architecture/architecture-brief.md

CANN/hccl仓软件架构展开



- **通信算子：**对上层AI框架提供统一的算子调用入口，内部完成集合通信算子的软件实现
- **算法（Executor）选择器：**根据当前**集群拓扑**和**硬件形态**，选择最佳性能的集合通信算法，返回对应的Executor名称
- **Executor层：**每一个Executor都提供了一个完整的算法实现，包括计算算法所需资源、计算通信关系。Executor可以组合多个基础算法（Template）来实现分级算法功能，**灵活应对昇腾950及之后产品的多样化产品形态**。
- **Template层：**实现算子实际任务编排逻辑，调用基础通信API，实现基础的集合通信算法

https://gitcode.com/cann

CANN

目录

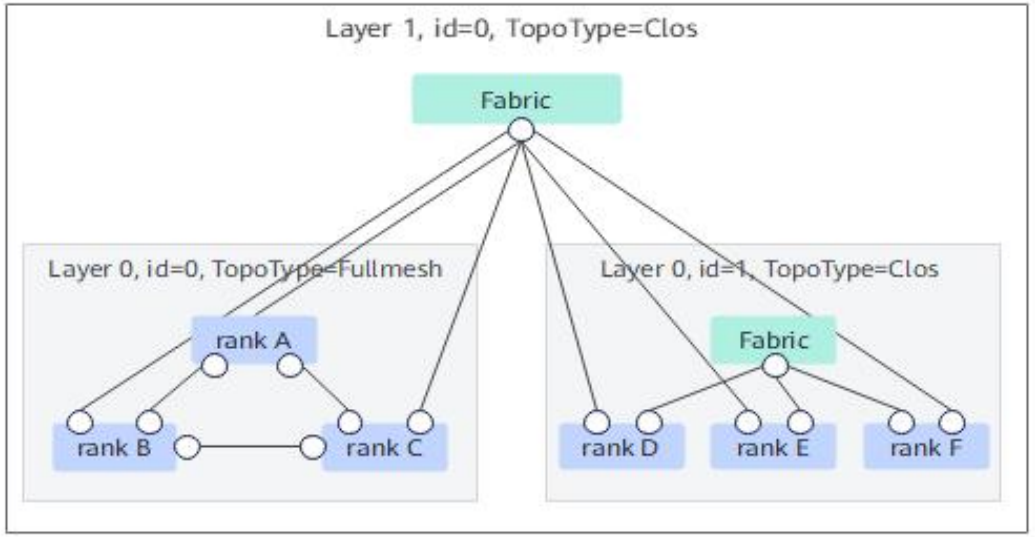
Part 1 HCCL软件架构

Part 2 HCCL算子编程模型

Part 3 HCCL算子常用编程API介绍

HCCL关键概念介绍——拓扑表达

拓扑模型：以Graph的方式对分级拓扑进行建模，并提供查询接口



类型	API	说明
拓扑感知	HcclRankGraphGetLayers	查询拓扑层级信息。
	HcclRankGraphGetRanksByLayer	查询本rank所在拓扑实例的rank信息。
	HcclRankGraphGetRankSizeByLayer	查询本rank在特定拓扑层级下的rank数量。
	HcclRankGraphGetTopoTypeByLayer	查询本rank在特定拓扑层级下的拓扑类型。
	HcclRankGraphGetInstSizeListByLayer	查询给定拓扑层级下的拓扑实例信息。
	HcclRankGraphGetLinks	查询特定拓扑层级下源rank和目的rank之间的连接信息。

- 每个通信域都有自己的一个RankGraph对象
- Node：Graph中的节点，包含两种类型：
  - 通信实体：在通信域中以rank ID标识的实体。
  - Fabric：对网络交换/路由的抽象。
    - Fabric只能与通信实体相连，通信实体可以通过Fabric两两互通；
    - Fabric可以是一个或多个交换设备组成的网络设施；
- Endpoint：一个Node的逻辑通信设备；
- Edge：Graph中的边，连接不同Node。
- Link：表示两个通信实体之间可以建链的信息，包含两端的Endpoint。
- 拓扑层级：实际网络拓扑存在分级情况，比如Server内拓扑、超节点内拓扑。

CANN

HCCL关键概念介绍——Memory

Input Buffer

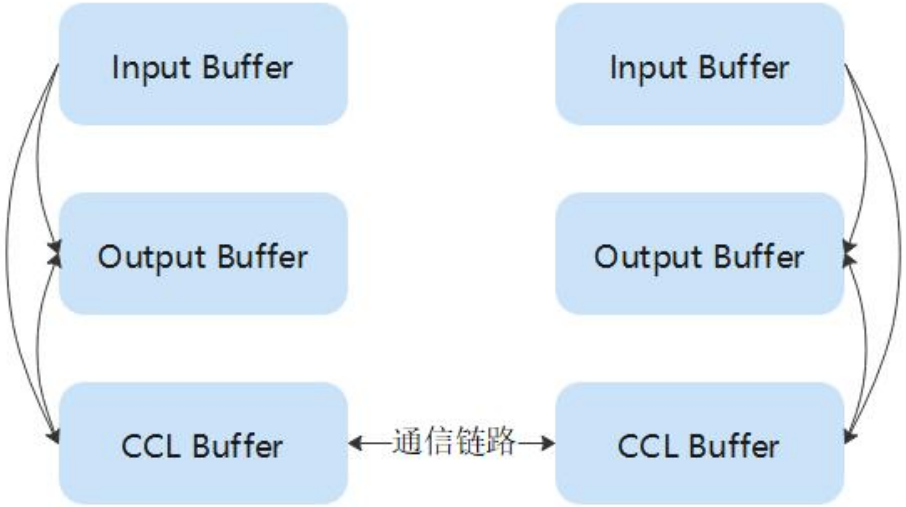
- 集合通信算子输入数据缓冲区。

Output Buffer

- 集合通信算子输出数据缓冲区，存放算法计算结果。

CCL Buffer

- CCL buffer是一组地址固定的Buffer，可被远端访问。单算子模式下，通信对象通过CCL Buffer来实现跨Rank的数据交换。
- CCL buffer和通信域绑定，通信域初始化的时候创建CCL buffer，默认大小是2*200M Byte，可以通过环境变量HCCL_BUFFSIZE进行修改（2*HCCL_BUFFSIZE）。
- 同一个通信域内执行的集合通信算子都复用相同的CCL Buffer。

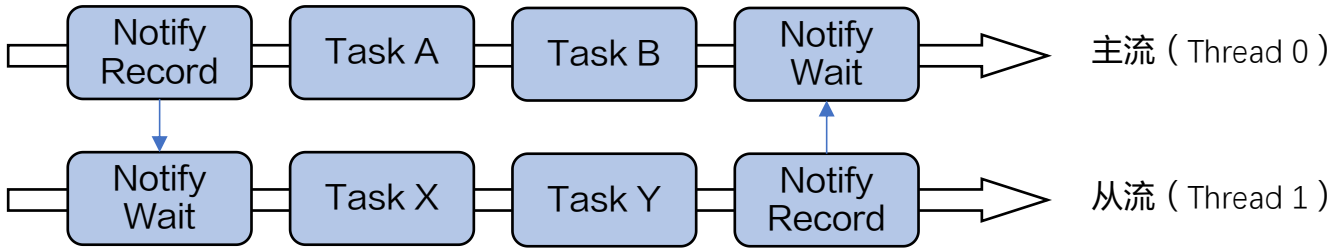


HCCL关键概念介绍——Thread

- 流（Stream）是NPU上的一种硬件资源，承载了待执行的Task序列。Task可以是一个DMA操作、一个同步操作或者一个NPU算子等。同一个流上的Task序列按顺序执行，不同流上的Task序列可并发执行。
- **HCCL软件将一个流抽象为一个Thread。**

主流
由Pytorch等训练框架调用集合通信API传入的stream对象称为主流。

从流
为了实现集合通信算法所需要的并行性而额外申请的stream对象称为从流。



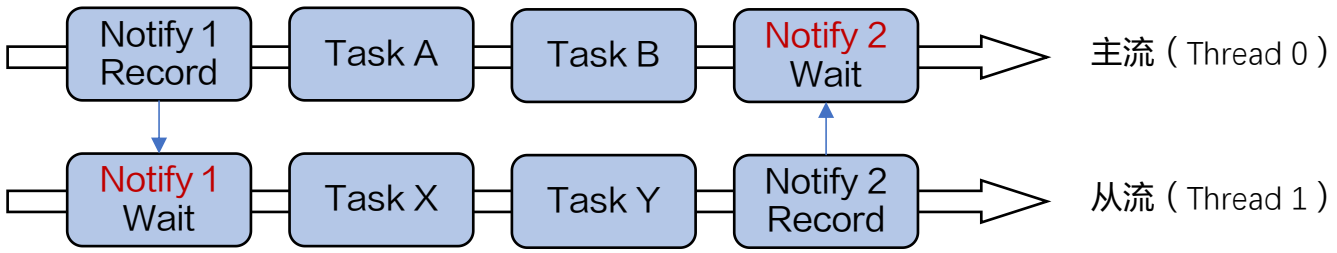
- 上图展示了一条主流和一条从流，主从流之间通过NotifyRecord/NotifyWait这一组Task进行同步。
- 主从流之间没有依赖关系时，Task可并行执行。如上图主流的TaskA、TaskB和从流的TaskX、TaskY可以是并行执行的。
- ThreadRecord/ThreadWait的具体含义在后文给出。

HCCL关键概念介绍——Notify

Notify是NPU上的硬件资源，用来做同步。在集合通信中主要有两种作用：
1）Rank内主从流之间的同步；
2）跨Rank数据收发的同步。

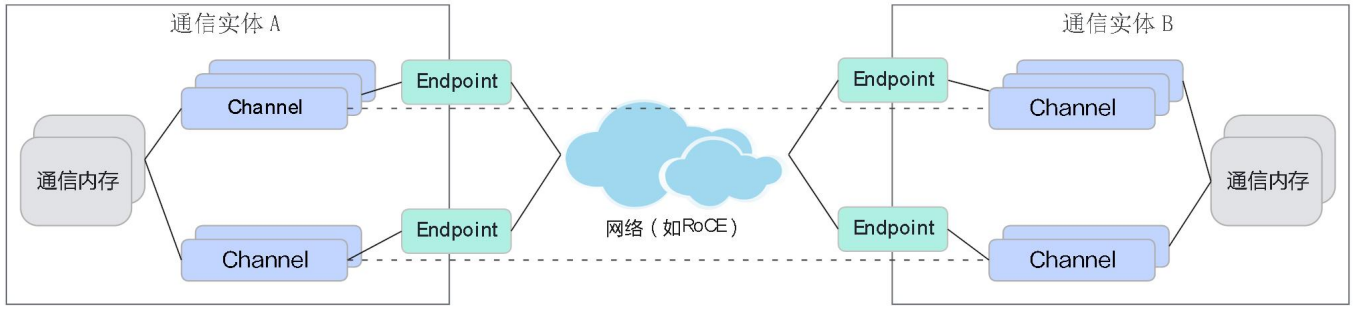
和Notify有关的Task有两大类：**Record** 和 **Wait**。
1）Record操作给对应的notify寄存器置1，并返回。如果对应的notify值已经是1，则不产生变化，直接返回。
2）Wait操作会等待对应的notify值变为1。条件满足后，将对应的notify值复位为0，并继续执行后续的Task。

Notify应用于主从流同步的示例：



主流通知从流，实质是将Notify1置位为1。从流通知主流，实质是将Notify2置位为1。

HCCL关键概念介绍——Channel



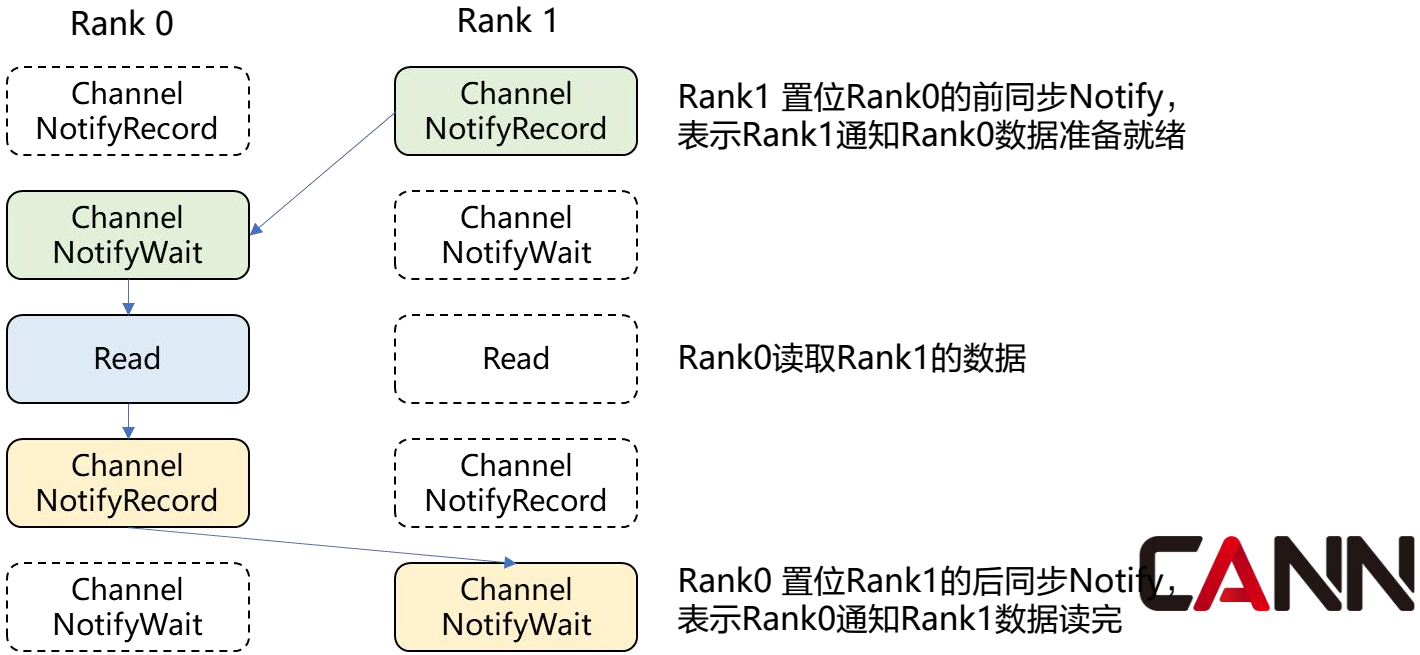
- 通信内存：注册到通信域、可被通信设备(Endpoint)访问的内存段；
- Endpoint：通信的逻辑设备，包含通信协议与通信地址
- Channel：通信实体间基于Endpoint创建的通信通道

要完成Rank间的数据通信需要先建立Channel（通信通道）。
Channel的两端一般各使用**2个**notify，分别称为**前同步Notify、后同步Notify。**
每条Channel的两端会申请各自的notify资源，不同的Channel之间不会复用notify。

基于Channel的数据收发及同步举例：

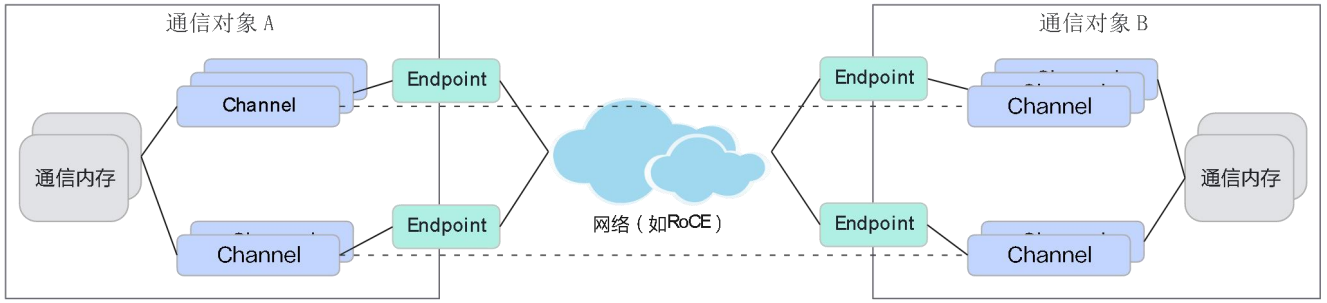
一次数据收发需要两组同步，如右图所示，分别使用了前同步和后同步两个notify。

为了避免同一条链路上多次收发数据相互影响，要求每次数据收/发前后需要分别进行前/后同步。



HCCL编程模型总结

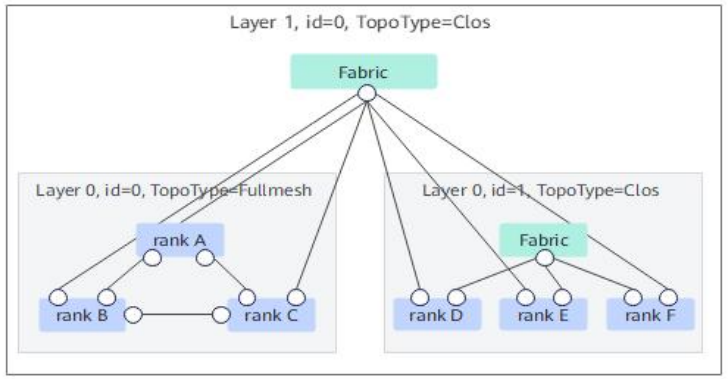
通信模型



Channel：

- 本端通信对象基于特定Endpoint与远端通信对象特定Endpoint之间建立的通信通道；
- 用户使用Channel可以读写远端通信对象内存或与远端通信对象进行同步。

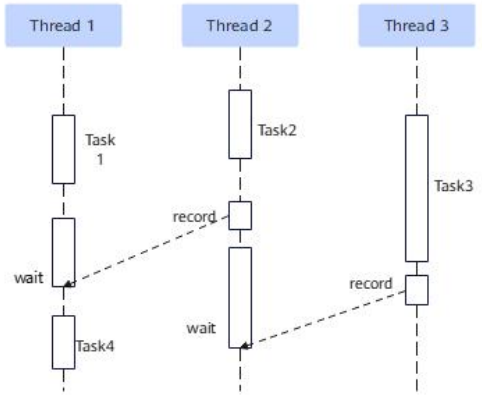
拓扑模型



- 以**RankGraph**的方式对分级拓扑进行建模，并提供查询接口

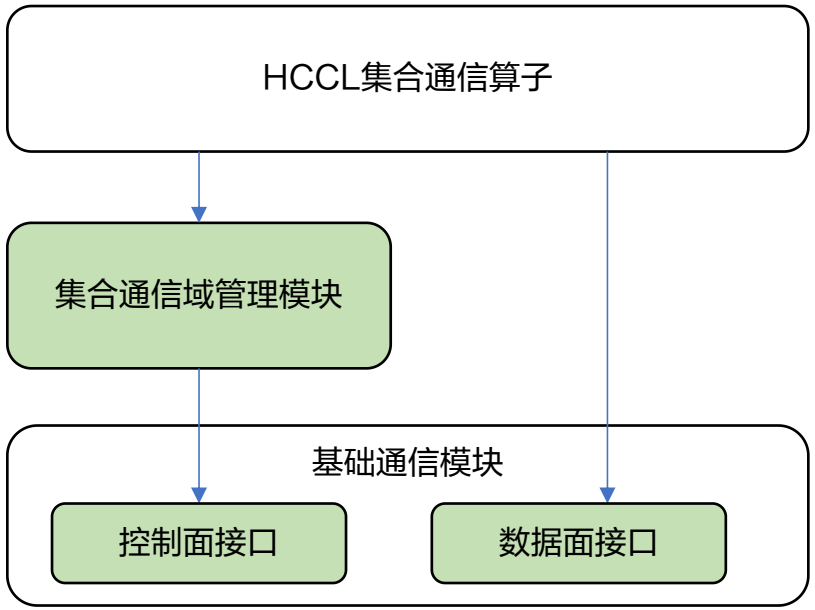
https://gitcode.com/cann

并发模型



- 并发单元抽象为**Thread**，通信任务与Thread绑定，不同Thread上的任务可并发执行
- Thread包含多个**Notify**，Thread间可以通过Notify进行同步；

通信算子编程API层次



CANN

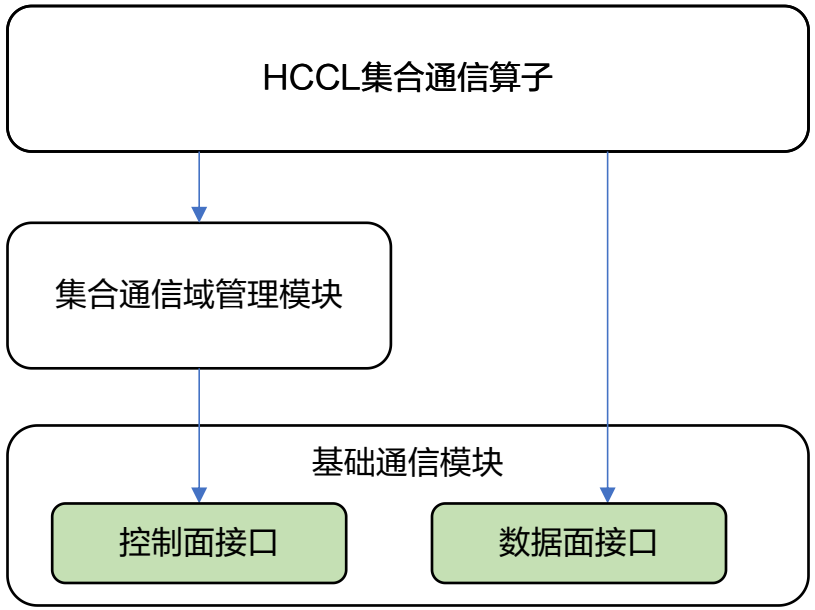
目录

Part 1 HCCL软件架构

Part 2 HCCL算子编程模型

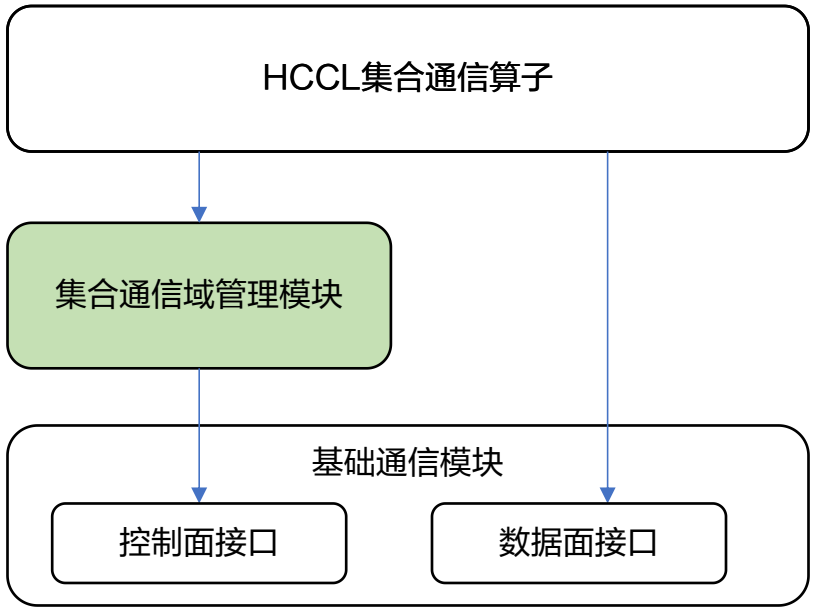
Part 3 HCCL算子常用编程API介绍

HCCL算子开发：通信算子编程API



类型		API	说明
控制面接口		HcommEndpointCreate	创建Endpoint
		HcommEndpointDestroy	销毁Endpoint
		HcommMemReg	内存注册
		HcommMemUnreg	内存解注册
		HcommMemExport	导出注册内存，用于交换
		HcommMemImport	导入远端交换的注册内存
		HcommChannelCreate	创建Channel
		HcommChannelDestroy	销毁Channel
		HcommThreadAlloc	申请Thread资源
		HcommThreadFree	释放Thread资源
数据面接口	本地操作	HcommLocalCopyOnThread	本地拷贝
		HcommLocalReduceOnThread	本地规约操作
		HcommThreadNotifyRecordOnThread	向其他Thread发送同步信号
		HcommThreadNotifyWaitOnThread	等待同步信号
	通信操作	HcommWriteOnThread	写远端内存操作
		HcommWriteReduceOnThread	写远端内存并做规约操作
		HcommReadOnThread	读远端内存操作
		HcommReadReduceOnThread	读远端内存并与本地做规约操作
		HcommChannelNotifyRecordOnThread	向远端发送同步信号
		HcommChannelNotifyWaitOnThread	等待远端同步信号
		HcommWriteWithNotifyOnThread	写远端内存并发送同步信号
		HcommWriteReduceWithNotifyOnThread	写远端内存并做规约操作，同时发送同步信号

HCCL算子开发：通信算子编程API



类型	API	说明
拓扑	HcclRankGraphGetLayers	查询拓扑层级信息。
	HcclRankGraphGetRanksByLayer	查询本rank所在拓扑实例的rank信息
	HcclRankGraphGetRankSizeByLayer	查询本rank在特定拓扑层级下的rank数量
	HcclRankGraphGetTopoTypeByLayer	查询本rank在特定拓扑层级下的拓扑类型
	HcclRankGraphGetInstSizeListByLayer	查询给定拓扑层级下的拓扑实例信息
	HcclRankGraphGetLinks	查询特定拓扑层级下源rank和目的rank之间的连接信息
Thread	HcclThreadAcquire	基于通信域获取Thread
	HcclThreadAcquireWithStream	
Channel	HcclChannelAcquire	基于通信域获取Channel
	HcclChannelGetHcclBuffer	基于Channel获取远端HCCL Buffer
通信上下文	HcclEngineCtxCreate	创建通信上下文资源
	HcclEngineCtxGet	查询通信上下文资源
	HcclEngineCtxCopy	将Host侧数据拷贝至通信上下文中
	HcclEngineCtxDestroy	销毁通信上下文资源
通信内存	HcclGetHcclBuffer	基于通信域获取HCCL Buffer
	HcclCommMemReg	基于通信域注册内存
	HcclChannelGetRemoteMems	获取Channel中交换的远端内存

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN